

Exercício 1

Nesta disciplina, adotamos a noção de “agente inteligente” proposta pelo livro AIMA de Russel e Norvig.

PARTE A. Defina essa noção em termos de *pensar* ou *agir* e *racional* ou *humano*.

PARTE B. Explique como podemos verificar se um agente computacional é inteligente ou não.

Resposta:

A) O agente inteligente tem o foco no comportamento externo, baseado numa ação e não em processos internos de pensamento ligados a cognição. O agente é definido pelo que faz: o ambiente alimenta seus sensores e ele retorna uma ação com seus atuadores.

O objetivo do agente não é imitar o comportamento humano, mas sim a racionalidade. Um agente inteligente é aquele que age para alcançar o melhor resultado possível ou o melhor resultado esperado, dadas as informações que possui. Um agente inteligente é um sistema que percebe seu ambiente e toma ações que maximizam suas chances de sucesso.

B) Para verificar se um agente computacional é inteligente, medimos sua racionalidade técnica e desempenho. Essa medida é feita através de um critério de sucesso. O mapeamento de percepção do agente é feito pela verificação da escolha da ação que maximiza o desempenho com base na sua sequência de percepções e no seu conhecimento prévio.

Diferente do teste de Turing, que foca na capacidade de se passar por humano, a verificação dos agentes é funcional. Se ele escolhe consistentemente o melhor resultado ele é considerado racional e, portanto, inteligente.

Exercício 3

Um agente do tipo consulta-tabela (lookup table) olha uma tabela do tipo percepção/ação (em oposição a um agente que raciocina ou que resolve problemas, como por exemplo um agente de busca). Esse tipo de agente pode enfrentar algumas dificuldades com relação à sua capacidade de resolver problemas e seu desempenho. Dificuldades também podem surgir durante a fase de projeto. Liste quais podem ser essas dificuldades.

Resposta:

Um agente de consulta-tabela armazena uma ação para cada sequência de percepções possíveis. Por conta desse comportamento, enfrenta algumas dificuldades:

- Tabela muito grande: o número de sequências de percepções possíveis cresce exponencialmente, mesmo para casos simples.
- Tempo de construção: é necessário definir manualmente a ação correta para cada combinação possível de entrada, o que é humanamente impossível em ambientes complexos.
- Falta de autonomia: não "aprende" nem se adapta, se o ambiente mudar ligeiramente e gerar uma percepção que não está na tabela, ele não saberá o que fazer. Depende inteiramente do conhecimento prévio inserido.
- Desperdício de memória: armazenar todas as combinações exige muito espaço na memória e o custo disso seria altíssimo.
- Não funciona com ambientes dinâmicos: não consegue prever consequências de ações que não foram explicitamente mapeadas.

Exercício 4

Explique as seguintes categorizações dos algoritmos de busca:

- (a) Busca informada e Busca não-informada
- (b) Busca em árvore e busca em grafo
- (c) Busca por planejamento de ações e Busca por identificação de estados
- (d) Busca local e Busca CSP
- (e) Busca online e Busca offline
- (f) Busca ótima e Busca completa

Resposta:

a) Busca informada (heurística): o agente utiliza conhecimento específico através de uma função heurística ($h(n)$) que estima o custo do estado atual até o objetivo. Isso permite priorizar os caminhos mais promissores (A^* , Greedy Best-First Search).

Busca não-informada: o agente não possui conhecimento sobre quão longe está do objetivo. Ele explora o espaço de estados de forma sistemática seguindo apenas a definição do problema (BFS, DFS, Busca de Custo Uniforme).

b) Busca em árvore: explora o espaço de estados sem manter um registro dos estados já visitados. Isso pode levar a caminhos redundantes ou loops infinitos se o grafo original contiver ciclos.

Busca em grafo: inclui uma lista de itens explorados para garantir que o mesmo estado não seja expandido mais de uma vez. É mais eficiente em ambientes onde múltiplos caminhos levam ao mesmo estado, evitando trabalho redundante.

c) Busca por planejamento de ações: o foco é encontrar uma sequência de ações que leve do estado inicial ao objetivo. O caminho percorrido é a solução.

Busca por identificação de estados: utilizada quando o caminho não importa, apenas o estado final que satisfaz as condições do problema (Problema das 8 rainhas).

d) Busca local: trabalha com um único estado atual e tenta melhorar esse estado indo para posições vizinhas. É ideal para problemas de otimização onde o espaço de busca é muito grande para ser mapeado sistematicamente (Hill Climbing, Simulated Annealing).

Busca CSP: problemas de Satisfação de restrições - o estado é definido por um conjunto de variáveis que devem respeitar restrições específicas. A busca foca em encontrar uma atribuição de valores que não viole nenhuma regra (Sudoku, coloração de mapas).

e) Busca online: o agente intercala computação e ação. Ele expande estados à medida que os percebe no ambiente real, sendo necessária quando o ambiente é desconhecido ou dinâmico.

Busca offline: o agente calcula a solução completa localmente antes de realizar qualquer ação no ambiente real.

(f) Busca ótima: um algoritmo é ótimo se ele garante encontrar a solução com o menor custo possível (o melhor caminho). Nem toda busca completa é ótima (BFS encontra a solução, mas só é ótima se todos os custos de passo forem iguais).

Busca completa: um algoritmo é completo se ele garante encontrar uma solução caso ela exista.

Exercício 6

Considere o grafo abaixo em que os nós vermelhos indicam estados meta e o nó azul indica o estado inicial. **Considere que a ordem de geração dos nós sucessores segue a ordem alfabética das letras que rotulam os nós.**

Parte A. Represente a árvore de busca completa (terminando os caminhos na meta ou em algum estado sem ações aplicáveis) usando uma busca em árvore.

Parte B. Descreva a ordem em que os nós serão visitados para cada uma das estratégias listadas abaixo (um nó é visitado para checar se ele corresponde a um estado meta e, caso não seja, para gerar os seus sucessores). Termine a busca se **o nó visitado** corresponder a um estado meta.

- 4.1 Busca em largura em árvore
- 4.2 Busca em largura em grafo
- 4.3 Busca em profundidade em árvore
- 4.4 Busca em profundidade em grafo
- 4.5 Busca de profundidade iterativa (passo da profundidade = 1) em grafo

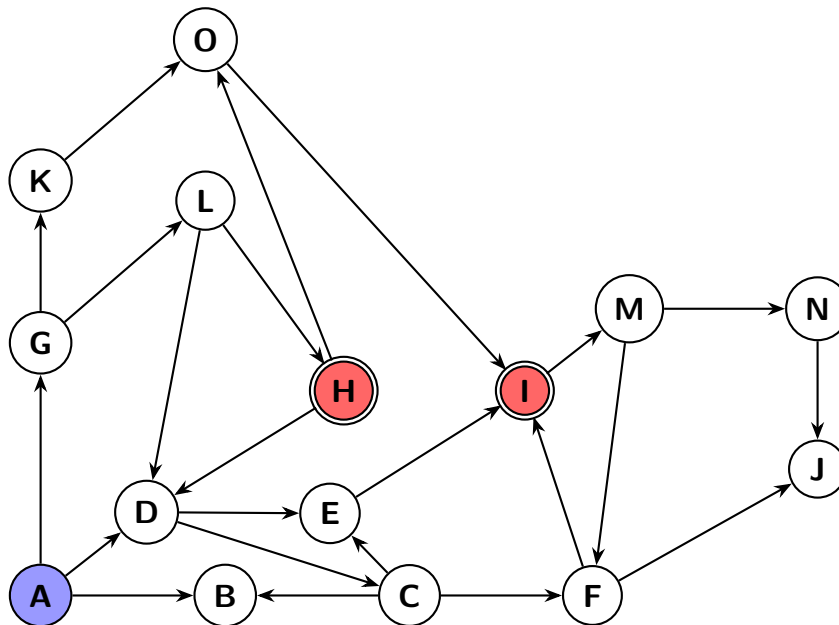
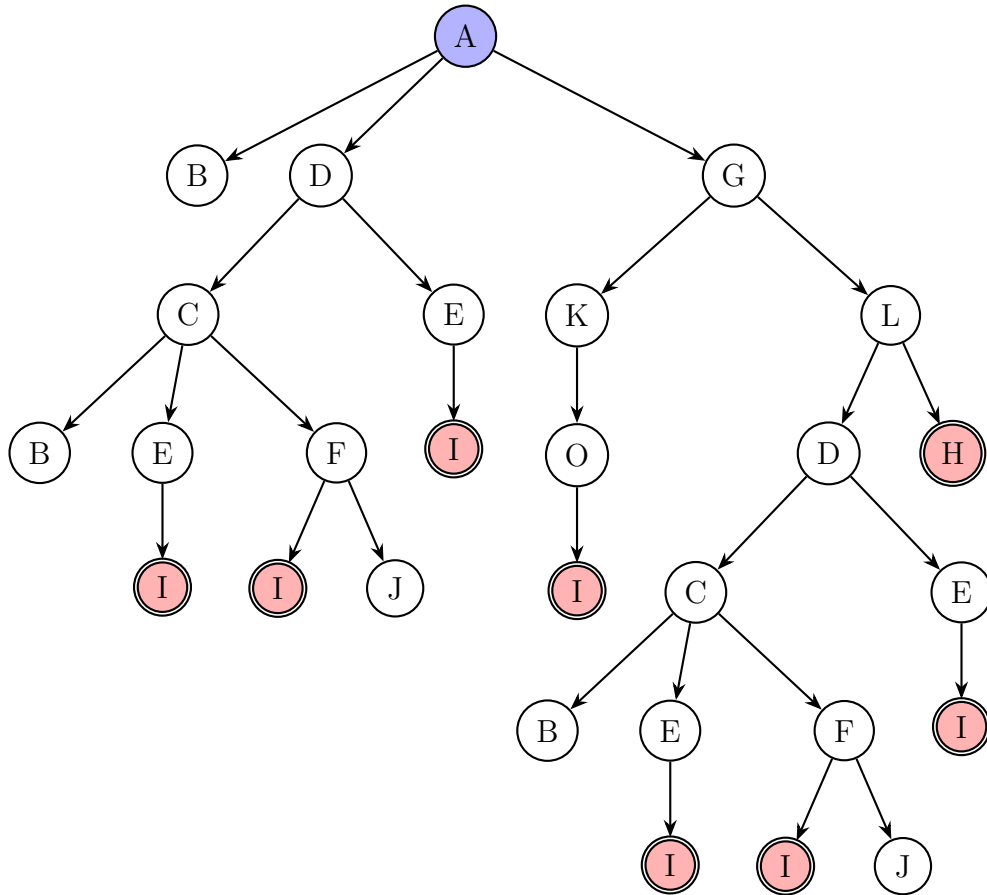


Figura 2. Espaço de estados.

Resposta:

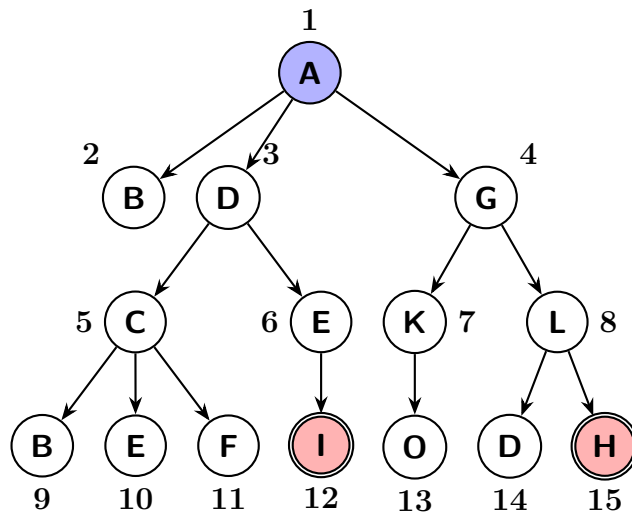
A) Em uma busca em árvore, não há verificação de estados repetidos. A árvore termina quando um nó é uma meta (H ou I) ou quando não há sucessores.



B)

4.1) Busca em largura em árvore

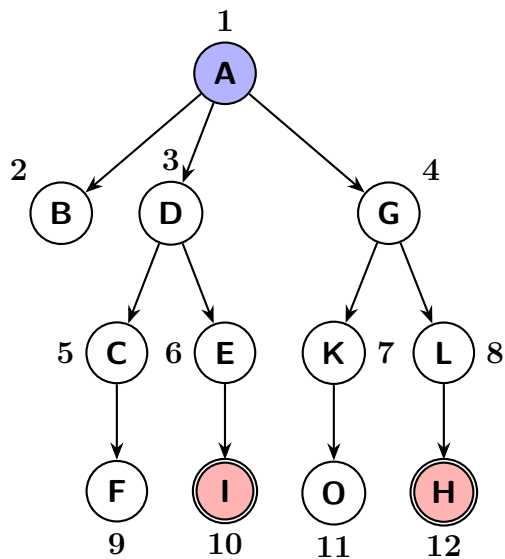
A árvore é expandida de forma completa nível por nível até que a meta H ou I seja extraída da borda. Os estados repetidos aparecem como novos nós. Como a ordem de geração é alfabética e a busca é em largura, todos os nós de um nível são explorados antes de passar para o próximo.



A busca só vai até o 12 e retorna o I. O restante é apenas a expansão, a consulta para na primeira meta encontrada.

4.2) Busca em largura em grafo

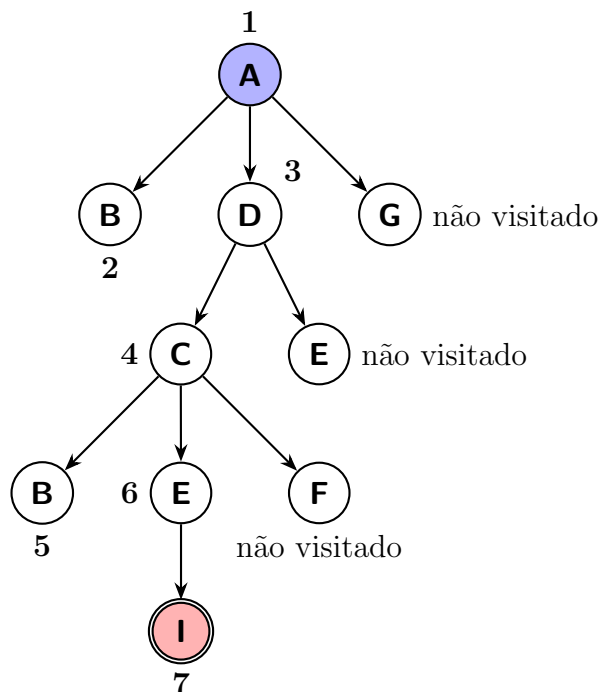
O algoritmo mantém um conjunto de estados explorados. Um nó não é adicionado à borda se o estado já tiver sido visitado ou se já estiver na borda.



A busca só vai até o 10 e retorna o I. O restante é apenas a expansão, a consulta para na primeira meta encontrada.

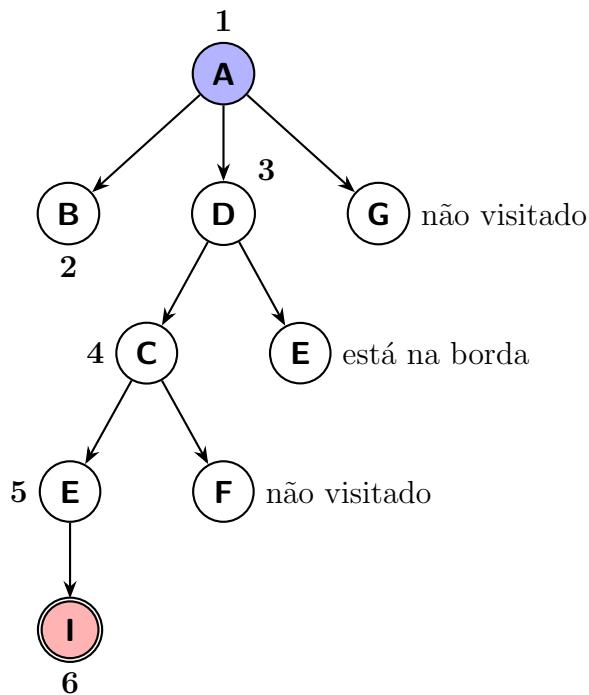
4.3) Busca em profundidade em árvore

O algoritmo explora o ramo mais à esquerda até o fim antes de fazer o backtracking. Não há memória de estados visitados.



4.4) Busca em profundidade em grafo

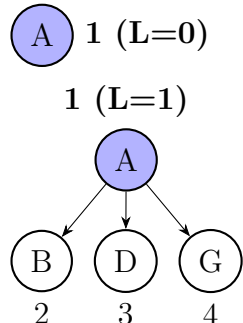
O algoritmo utiliza uma lista de estados explorados. Quando um nó é gerado, se o seu estado já foi visitado ou já está na borda, ele é descartado. O nó B não será visitado uma segunda vez quando a busca estiver explorando os filhos de C.

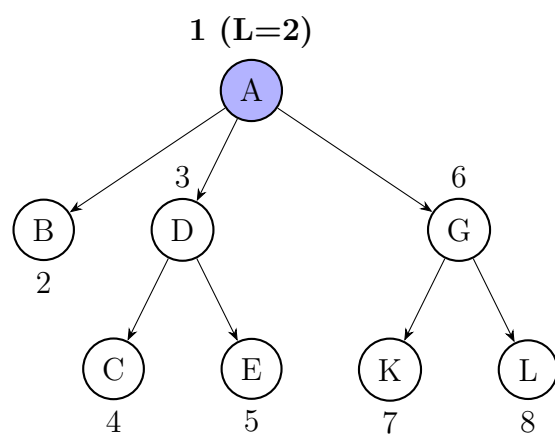


4.5) Busca de profundidade iterativa (passo da profundidade = 1) em grafo

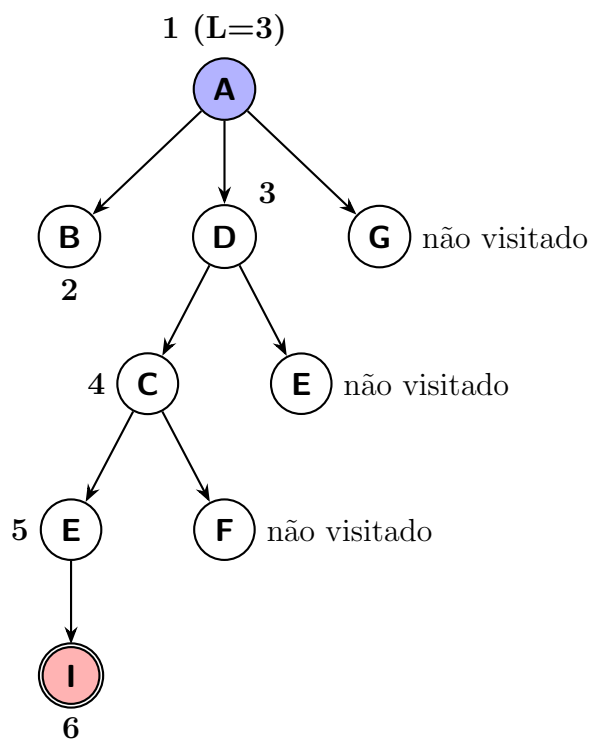
O algoritmo realiza múltiplas buscas em profundidade (DFS) com limites crescentes.

Casos de falha:



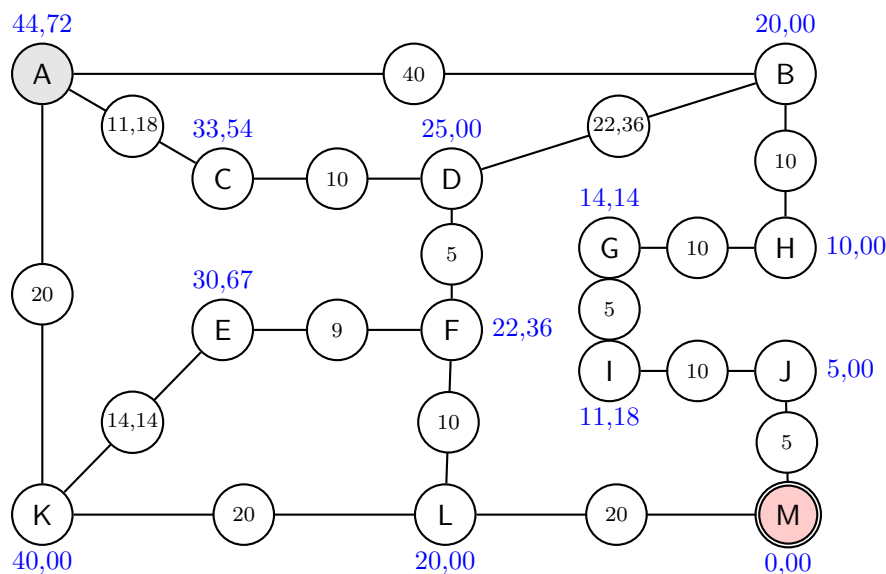


Caso de sucesso:



Exercício 10

Considere o problema de cálculo de rotas entre diferentes cidades. No grafo ilustrado da Figura 3, cada nó representa uma cidade distinta, e cada aresta, uma rodovia que interliga duas cidades, cujo peso indica a distância, em km, entre essas cidades por uma rodovia. Suponha que se deseje encontrar a melhor rota entre as cidades A e M, indicadas nesse grafo. Considere, ainda, os valores indicados na tabela abaixo como a distância em linha reta, em km, de cada cidade para a cidade M.



Resposta:

1. (V) Utilizando o algoritmo A^* , a rota entre A e M encontrada é ACDFLM e o custo do caminho é 56,18.

O algoritmo A^* escolhe o nó que minimiza $f(n) = g(n) + h(n)$.

O custo real (g) do caminho $A \rightarrow C \rightarrow D \rightarrow F \rightarrow L \rightarrow M$ é: $11,18 + 10 + 5 + 10 + 20 = \mathbf{56,18}$.

Comparando com outras rotas (como via K ou via B), este caminho apresenta o menor custo acumulado e a heurística é consistente, garantindo que o A^* encontre essa rota como a ótima.

2. (F) Utilizando a busca gulosa, a rota encontrada no problema acima é

ACDFLM.

A busca gulosa expande o nó com o menor valor de heurística $h(n)$.

De A, vizinhos são: $B(h = 20)$, $C(h = 33, 54)$, $K(h = 40)$. Escolhe B.

De B, vizinhos são: $F(h = 22, 36)$, $H(h = 10)$. Escolhe H.

De H, vizinhos são: $B(h = 20)$, $G(14, 14)$. Escolhe G.

De G, vizinhos são: $I(h = 11, 18)$. Escolhe I.

De I, vizinhos são: $J(h = 5, 00)$. Escolhe J.

De J, vizinhos são: $M(h = 0, 00)$. Escolhe M.

Assim a rota na busca gulosa é ABHGIJM.

3. (V) Utilizando-se a busca gulosa, a rota encontrada no problema acima é ABHGIJM.

Feito na 2.

4. (F) Para utilizar algoritmos de busca heurística, deve-se definir uma heurística que superestime o custo da solução.

É o contrário. Para garantir a otimalidade do A^* , a heurística deve ser admissível, o que significa que ela nunca pode superestimar o custo real até a meta ($h(n) \leq h^*(n)$).

5. (F) O A^* é um algoritmo ótimo e completo quando heurísticas admissíveis são utilizadas, independentemente se usamos busca em árvore ou em grafo.

A admissibilidade garante a otimalidade apenas na busca em árvore. Para busca em grafo, a admissibilidade não é suficiente para garantir a otimalidade. É necessário que a heurística seja consistente (ou que o algoritmo reabra nós fechados).

6. (V) O A^* é um algoritmo ótimo e completo quando heurísticas consistentes são utilizadas, independentemente se usamos busca em árvore ou em grafo.

A consistência garante que o $f(n)$ nunca diminua ao longo de um caminho. Isso assegura que, ao expandir um nó pela primeira vez em um grafo, o caminho encontrado até ele já é o ótimo.

7. (V) No algoritmo simulated annealing, é possível haver movimentos para um estado com avaliação pior do que a do estado corrente, dependendo da temperatura do processo e da probabilidade.

É um algoritmo de busca local projetado para escapar de máximos ou mínimos locais (pontos onde o algoritmo ficaria preso por não encontrar vizinhos melhores imediatos).

8. (F) Toda heurística admissível é consistente.

O inverso é verdadeiro (toda heurística consistente é admissível). Existem heurísticas que nunca superestimam o custo (admissíveis), mas que violam a desigualdade triangular entre nós sucessores, não sendo, portanto, consistentes.

Exercício 13

Considere duas heurísticas admissíveis $h_1(s)$ e $h_2(s)$. Prove que cada uma das seguintes heurísticas induzidas a partir delas é admissível ou forneça um contraexemplo caso não seja verdadeiro. Se necessário denote o custo exato à meta a partir de um estado s por $h^*(s)$ e assuma que toda ação possui custo positivo.

- a) $h_1 + h_2$
- b) $h_1 \times h_2$
- c) $\max\{h_1, h_2\}$
- d) $1 + \min\{h_1, h_2\}$
- e) $\alpha \times h_1 + \beta \times h_2$, com $\alpha \geq 0$, $\beta \geq 0$ e $\alpha + \beta = 1$

Resposta:

- a) $h_1 + h_2$

Não é admissível.

Proof. Considere um estado s onde o custo real até a meta é $h^*(s) = 10$.

Suponha $h_1(s) = 6$ e $h_2(s) = 7$.

Ambas são admissíveis individualmente ($6 \leq 10$ e $7 \leq 10$), mas a soma resulta em $h_1(s) + h_2(s) = 13$, o que supera o custo real $h^*(s) = 10$, violando a admissibilidade.

□

- b) $h_1 \times h_2$

Não é admissível.

Proof. Se $h^*(s) = 5$, $h_1(s) = 2$ e $h_2(s) = 3$ (ambas admissíveis), o produto é $h_1 \times h_2 = 6$.

Como $6 > 5$, a heurística superestima o custo real e não é admissível.

□

- c) $\max\{h_1, h_2\}$

É admissível.

Proof. Sabemos que $h_1(s) \leq h^*(s)$ e $h_2(s) \leq h^*(s)$, por definição de admissibilidade.

O valor máximo entre dois números que são ambos menores ou iguais a um terceiro, obrigatoriamente também será menor ou igual a esse terceiro: $\max\{h_1(s), h_2(s)\} \leq h^*(s)$.

□

d) $1 + \min\{h_1, h_2\}$

Não é admissível.

Proof. No estado meta g , por definição, o custo real é $h^*(g) = 0$ e qualquer heurística admissível deve retornar $h(g) = 0$.

Para esta função, teríamos $1 + \min\{0, 0\} = 1$.

Como $1 > 0$, ela superestima o custo no estado meta e falha na admissibilidade.

□

e) $\alpha \times h_1 + \beta \times h_2$, com $\alpha, \beta \geq 0$ e $\alpha + \beta = 1$

É admissível.

Proof. Esta é uma média ponderada de duas funções admissíveis.

Como $h_1(s) \leq h^*(s)$ e $h_2(s) \leq h^*(s)$, temos:

$$\begin{aligned}\alpha \times h_1(s) + \beta \times h_2(s) &\leq \alpha \times h^*(s) + \beta \times h^*(s) \\ (\alpha + \beta) \times h^*(s) &= 1 \times h^*(s) = h^*(s)\end{aligned}$$

Como o resultado é sempre menor ou igual a $h^*(s)$, a combinação permanece admissível.

□